

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Paralelne i distribuirane arhitekture i jezici

Vlasništvo

Branislav Ristić

Zimski semestar, školska 2026./27.

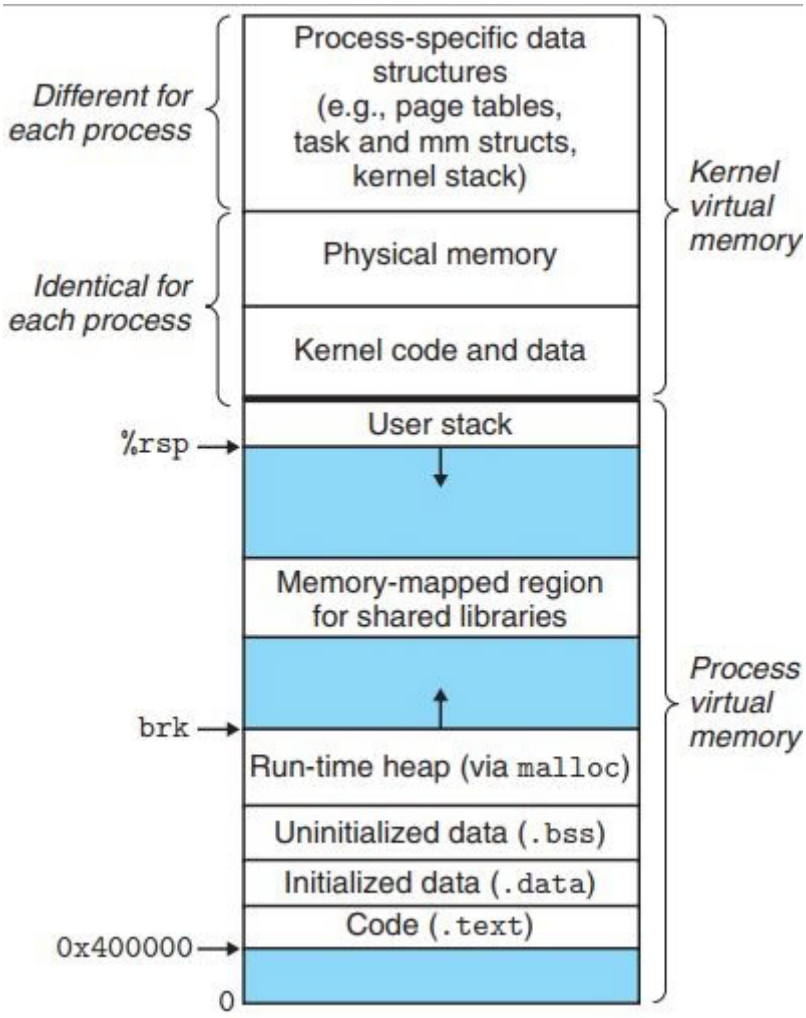
Memorija

Stack vs. Heap

Osobina	Stack	Heap
Struktura	LIFO (Last In, First Out)	Bez određenog redosleda
Veličina	Fiksna, manja	Dinamička, veća
Alokacija	Automatska (brza)	Preko `Box`, `Vec`, `String` (sporija)
Dealokacija	Automatska pri izlasku iz opsega	Automatska (`Drop`), kad vlasnik izađe iz opsega
Životni vek	Ograničen na opseg	Dok postoji vlasnik
Brzina pristupa	Brži	Sporiji
Niti	Svaka nit ima svoj stack	Deljen između niti, potrebna sinhronizacija
Tipična upotreba	Pozivi funkcija, lokalne promenljive, rekurzija	Dinamičke strukture, velike alokacije

Virtuelna memorija

- Svaki proces ima svoj virtuelni adresni prostor: u njemu su kod, stack i heap.
- Memorija procesa:
  - Stranice,
  - Okviri.
- Jedna od ključnih funkcionalnosti operativnog sistema:
  - Izolovanost,
  - Bezbednost.



Slika 1: Raspored virtuelne memorije procesa

Vlasništvo

Vlasništvo

- Funkcionalnost karakteristična za Rust programski jezik.
- Duboke implikacije na rad jezika.
- Omogućava memorijsku sigurnost bez GC-a.
- Predstavlja skup pravila koja govore kako upravljati memorijom.
- Primer:

primeri/ownership/src/bin/smart_pointers.cpp

```
#include <iostream>
#include <memory>
#include <utility>

using namespace std;

class Nesto{
    public:
        Nesto(){
            cout << "Konstruktor klase Nesto na adresi " << this << endl;
        }
```

```
        ~Nesto(){
            cout << "Destruktor klase Nesto na adresi " << this << endl;
        }
        void akcija(){
            cout << "Neka akcija klase Nesto na adresi " << this << endl;
        }
};


struct Struktura{
        Nesto* goli;
        unique_ptr<Nesto> pametan;
        shared_ptr<Nesto> deljen;
};

int main(){
    cout << "Program pocinje" << endl;
    /*
        Ovaj primer demonstrira rad sa običnim pokazivačem na kojim ste
 ↪  navikli.
        Problem sa takvim pokazivačem jeste da morate da pamtite da ga
 ↪  obrišete, prvo
        i drugo, on može biti smešten bilo gde, u bilo koju lokaciju, bilo
 ↪  kada u
        bilo kom broju primera što može da dovede do toga da se obriše na
 ↪  više mesta
        (greška), ne obriše nigde (greška), ili koristi mnogo pošto je
 ↪  obrisan
        (naročito loša greška).

        Zbog toga se upotreba ovakvih, 'golih' pokazivača više ne
 ↪  preporučuje u
        produkcionom kodu.
    */
    cout << "Primer #1: 'Go' pointer" << endl;
    {
        Struktura s;
        {
            Nesto* n = new Nesto();
            s.goli = n;
            n->akcija();
            s.goli->akcija();
            delete n;
        }
        s.goli->akcija();//Ovaj kod, zastrašujuće, __neće izazvati
 ↪  grešku.__
        //Uprkos tome što je ovo pokazivač na nekorišćenu memoriju, podaci
         ↪  su
        //ostali i kod će se izvršiti. Možda. Ako je neko alocirao tu
         ↪  memoriju u međuvremenu
        //greške će biti. Ovo je užasan problem za debagovanje.
    }

    cout << "Primer #2: Jedinstveni pointer" << endl;
    /*
        Jedinstveni pokazivači, sa druge strane, nude mnoga poboljšanja.
 ↪  Prvo, jedinstveni su
```

```
        to znači da ne možete ni slučajno da ih kopirate. Jedinstvenost je
↪   garantovana i
        pokušaj konstruktora kopiranja ili operatora dodele izaziva
↪   grešku.

        Jedino što možete da uradite sa jedinstvenim pokazivačem jeste da
↪   ga pomerite, tj.
        da prestane da važi na jednom mestu i počne da važi na drugom.
↪   Zašto ovako?
        Zato što tako memorija ostaje uvek u poznatoj nadležnosti: nikad
↪   se neće desiti da
        ne znate čija je to memorija. Ovo je, takođe, način da vam, npr.
↪   funkcija vrati memoriju
        i da bude jasno da je vaš posao, kada dobijete tu memoriju, da je
↪   počistite.

        Još bolje, jedinstven pokazivač zna kada izađe iz opsega, tj, iz
↪   funkcije ili bloka u
        kome je generisan. Čim se to desi, on dealocira svoju memoriju
↪   istog trenutka.
        To znači da možete da se opustite oko delete operatora. Biće
↪   pozvan mesto vas,
        kada god vam je potreban.

        Veliku većinu vremena, pokazivači koje želite su baš ovakvi,
↪   jedinstveni
        pokazivači.
    */
    {
        Struktura s;
        {
            unique_ptr<Nesto> n(new Nesto());
            n->akcija();
            s.pametan = move(n);//Ovim se koriste takozvani konstruktori
↪   pomeranja
            //Da se n __premesti__ u s.pametan. To znači da n više ne
             ↪  postoji, efektno
            //i da je kontrola prebacena u s.pametan.
            s.pametan->akcija();
        }
        s.pametan->akcija();//Za razliku od prvog primera, ovo i treba da
↪   radi. Tek kada
        //s izađe iz opsega će destruktor biti pozvan, budući da je
         ↪  kontrola nad resursom
        //alociranim gore u liniji 80 prebačena u ruke strukture. Ovo je
         ↪  mala razdaljina
        //ovde u primeru, ali u ozbiljnom programu, to prebacivanje može
         ↪  ići bilo gde.
    }
    cout << "Primer #3: Deljen pointer" << endl;
    /*
        Deljeni pokazivači su kao i pametni sa jednom razlikom: mogu se
↪   kopirati slobodno,
        ali svaki pamti da je kopija i na centralnom mestu se broji koliko
↪   referenci ima
        na datu memoriju. Kako koji shared_ptr izlazi iz svog opsega,
↪   poziva svoj
```

```
        destruktor koji smanjuje globalni brojač referenci za 1. Kada on
 ↪  dostigne vrednot 0
        Dealocira se i memorija na koju pokazivač pokazuje.

        To znači da možete imati dva pokazivača na neku memoriju i
 ↪  osloboditi je tek kada su
        oba izbačena iz opsega. Ovo je jako moćan alat.
    */
    {
        Struktura s;
        {
            auto n = make_shared<Nesto>();//Parametri 'make_shared' su ono
             ↪  što bi išlo u
            //konstruktor klase 'Nesto.' Pozivanjem ovoga broj referenci
             ↪  je sada 1.
            s.deljen = n; //Kopira se ovde. Broj referenci je sada 2.
            n->akcija();
            s.deljen->akcija();
        } //Ovde se broj referenci smanjuje na 1, jer je 'n' izašao iz
         ↪  opsega.
        s.deljen->akcija();
    }//Destruktor se poziva tek ovde jer je broj referenci tek ovde 0
     ↪  izlaskom iz opsega
    //s, što izbacuje iz opsega sve njene elemente uključujući i 'deljen.'

    cout << "Gotov program." << endl;
    return 0;
}
```

Pravila vlasništva

- Svaka vrednost u Rust programu ima vlasnika.
- U jednom trenutku vrednost ima samo jednog vlasnika.
- Ukoliko vlasnik izađe iz opsega, vrednost se dropuje: na kraju bloka, obrnutim redosledom od deklaracije, a ne posle poslednje upotrebe.

String literal i string struktura

- String literal:
  - Sadržaj, te i veličina, je poznata u vreme kompajliranja.
  - Hard-kodovana direktno u .rodata sekciju programa.
  - Brz, ali nepromenljive veličine.
- String struktura:
  - Sadržaj ne mora biti poznat u vreme kompajliranja.
  - Spor, ali dinamičke veličine.

String literal i string struktura: primer

primeri/ownership/src/bin/strings.rs

```rust
fn main() {
    // Define the slice s, which refers to the string literal
    let s = "hello";
    println!("{}", s);
    // `s` is a `&str` (no `Drop`), it is valid until the end of the scope

    // Define string s, which is mutable
```

```
    let mut s = String::from("hello"); // Includes pointer, len, cap

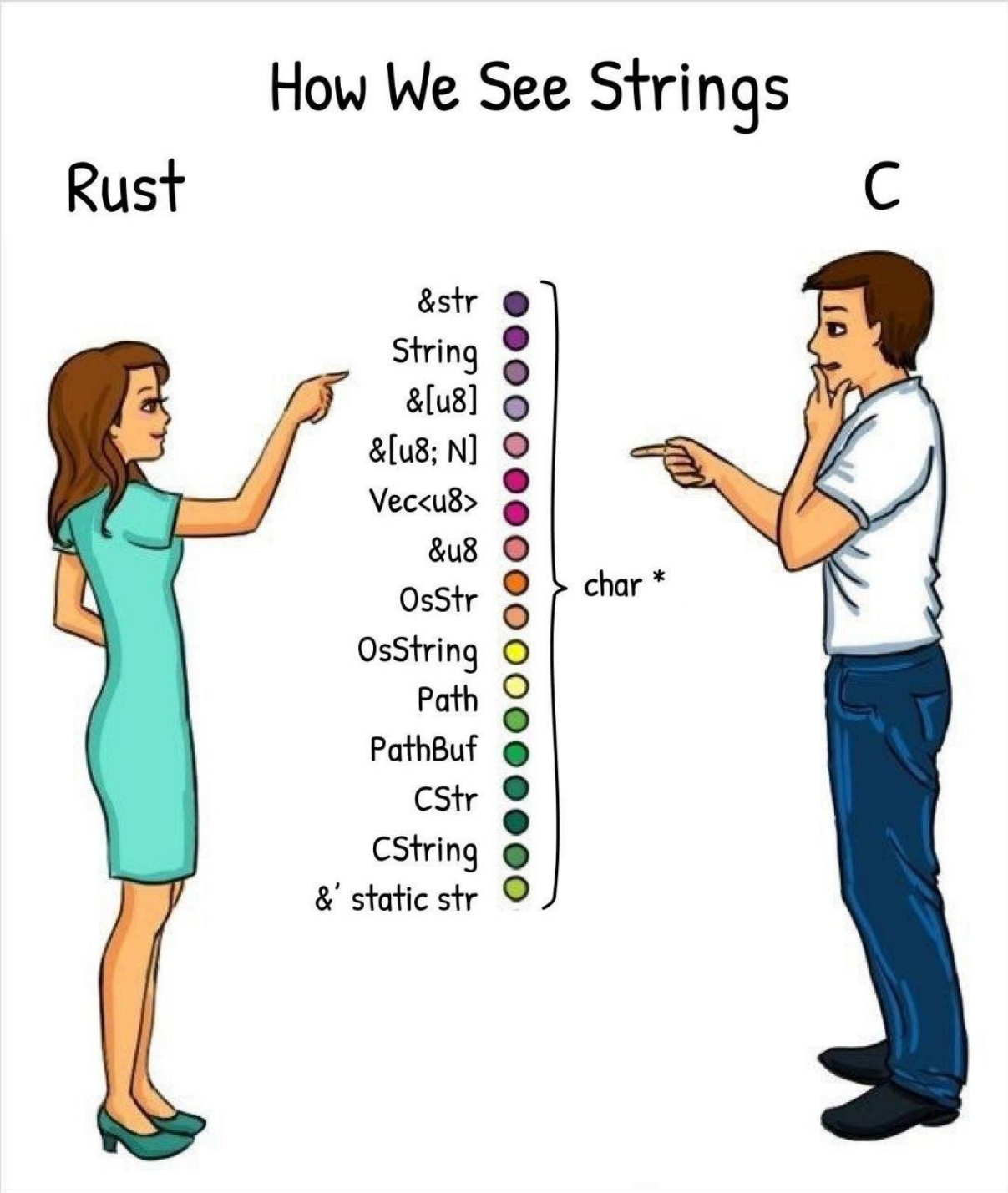
    // push_str() appends a literal to a String
    s.push_str(", world!");

    // This will print `hello, world!`
    println!("{s}");
}
```

Izlaz

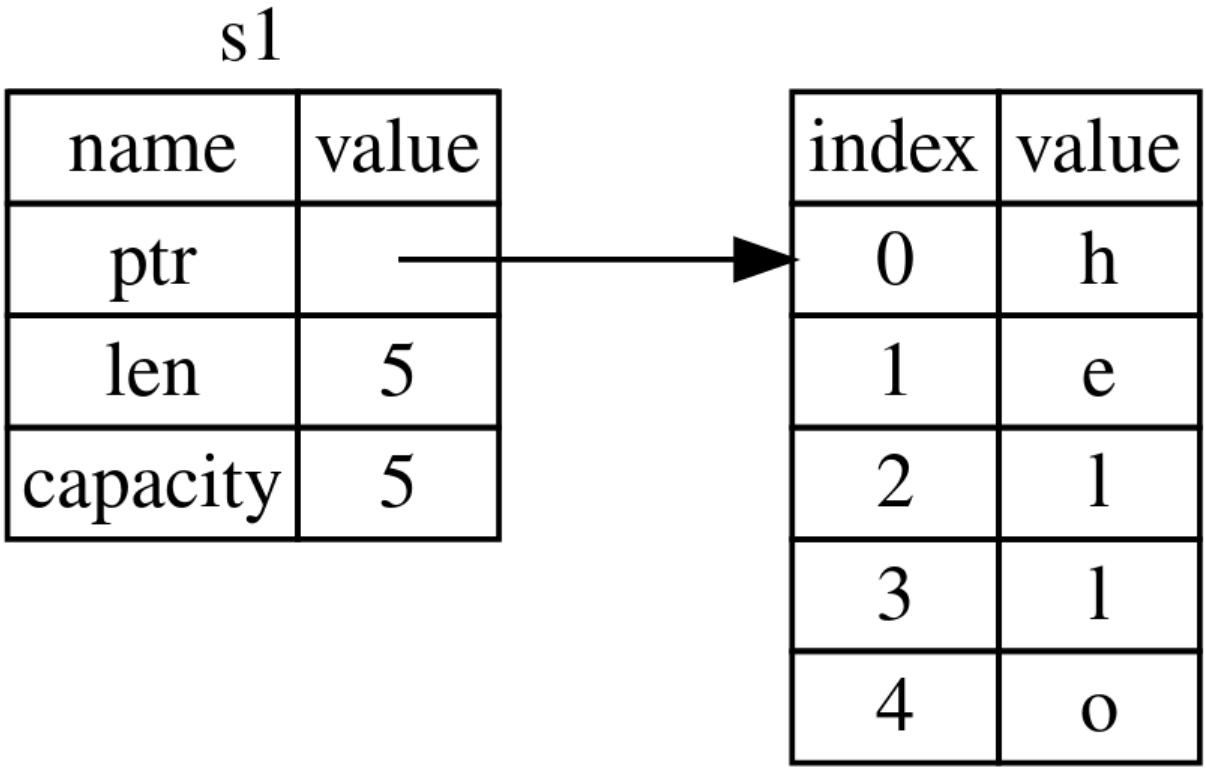
```
hello
hello, world!
```

Stringovi: Rust i C



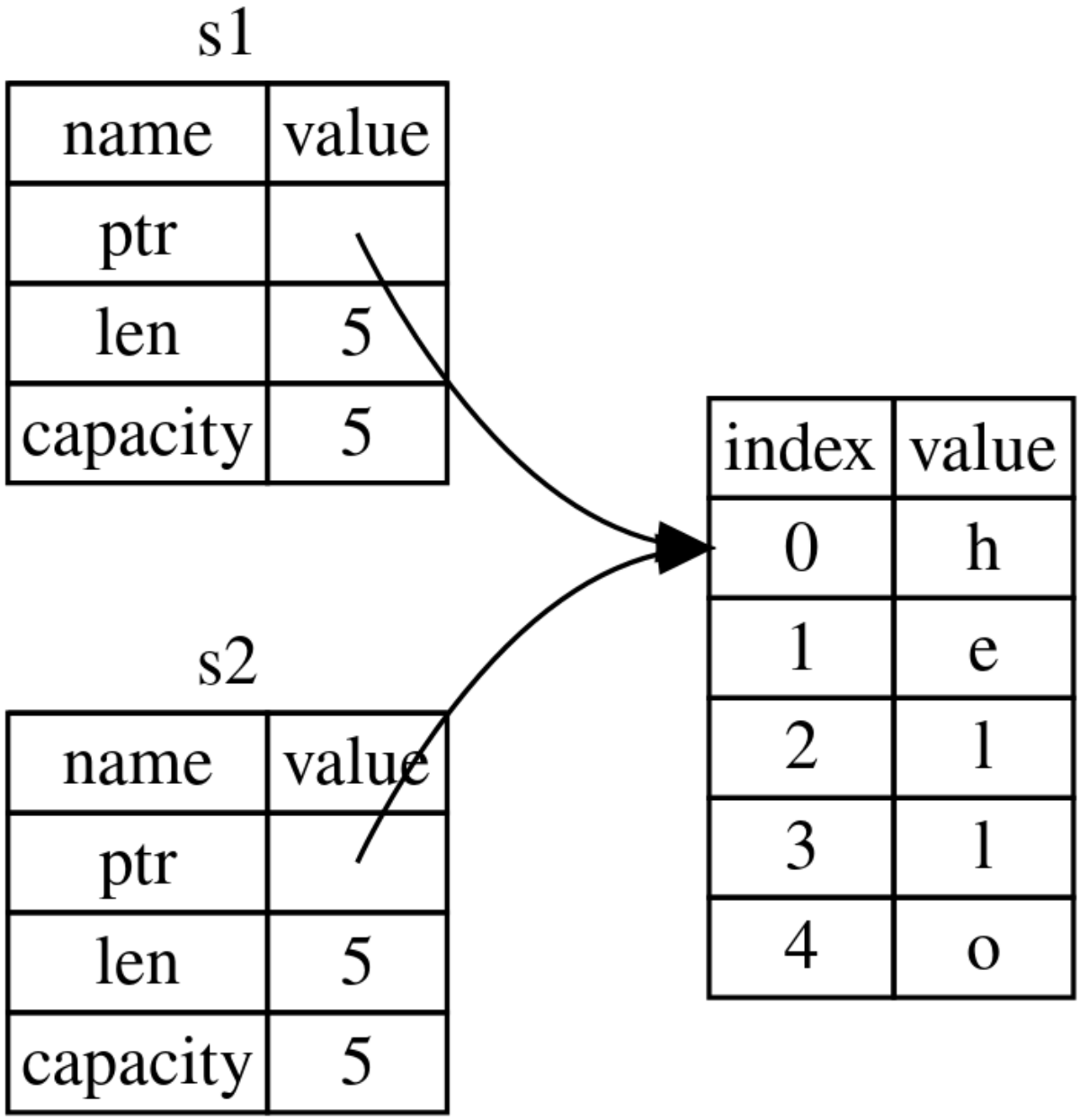
Slika 2: Kako Rust i C vide stringove

String: struktura u memoriji



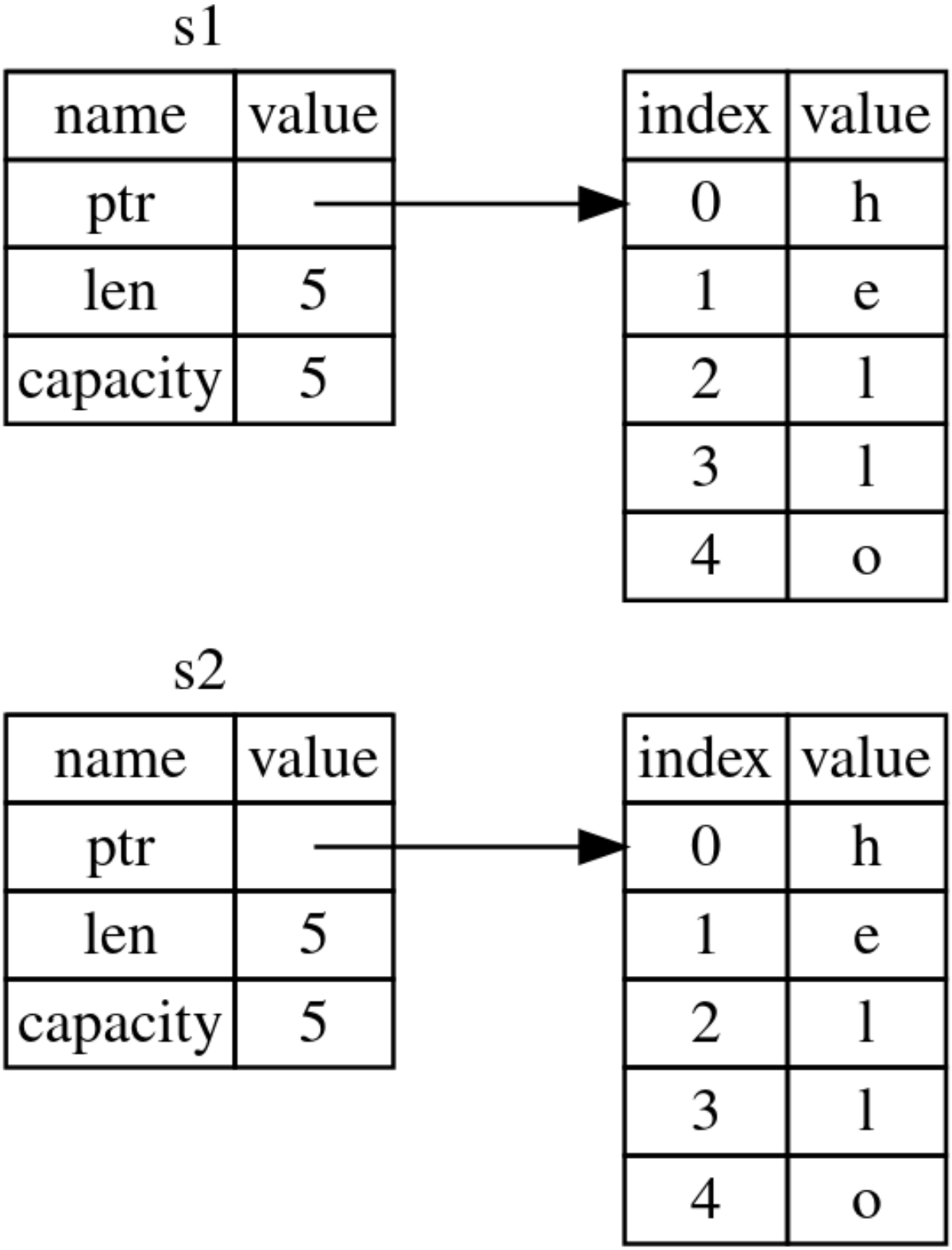
Slika 3: Struktura String tipa u memoriji

Shallow copy: isti podaci na heapu



Slika 4: Shallow copy: s1 i s2 pokazuju na iste podatke na heapu

Deep copy: odvojene kopije podataka



Slika 5: Deep copy: s1 i s2 imaju svoje kopije podataka na heapu

Deep copy: clone()

`primeri/ownership/src/bin/strings_clone.rs`

```rust
fn main() {
    let s1 = String::from("hello");

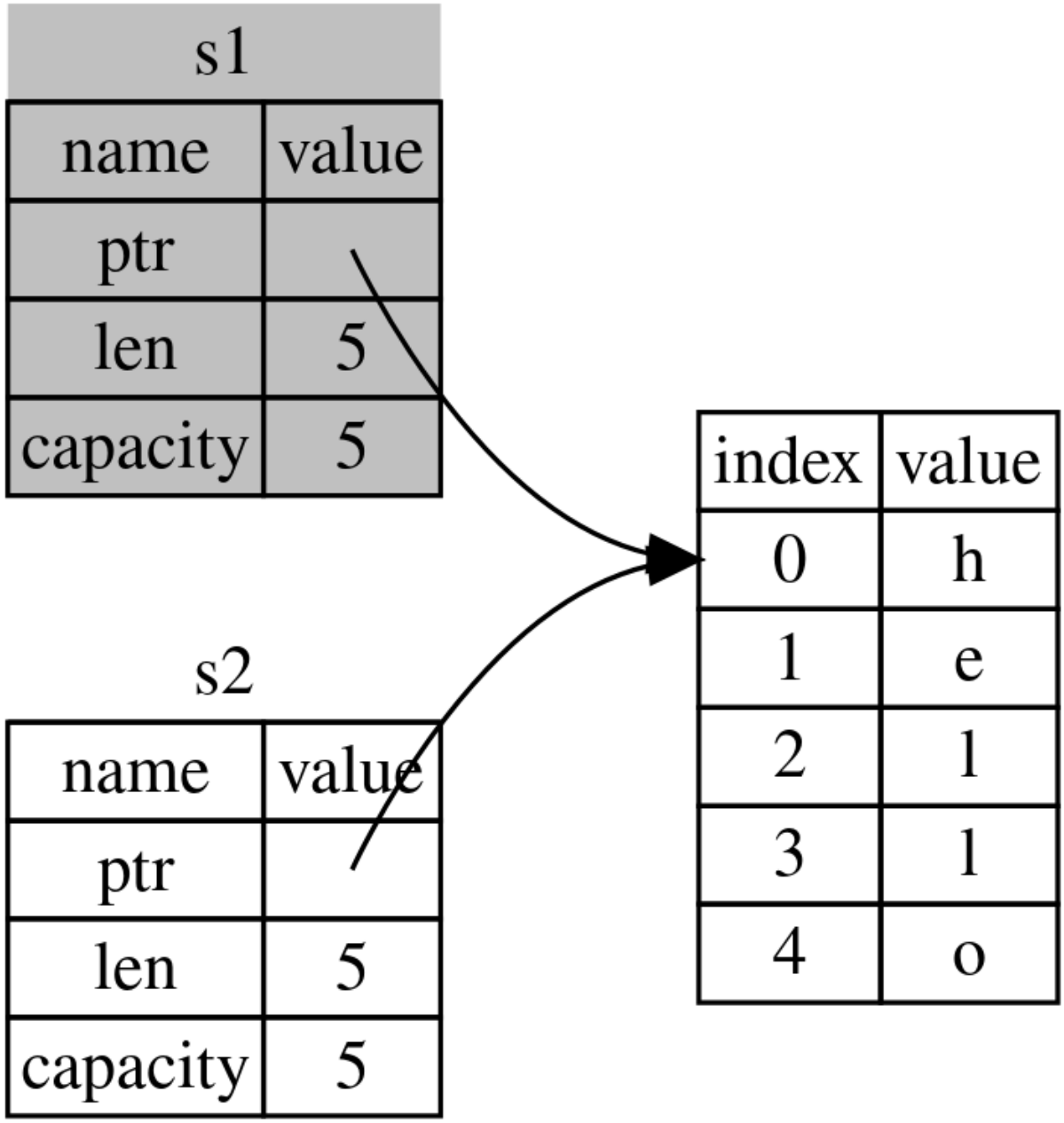
    let s2 = s1.clone(); // Deep copy: new heap allocation with the same
     ↪  data

    println!("s1 = {s1}, s2 = {s2}");
    println!("same heap data: {}", s1.as_ptr() == s2.as_ptr());
}
```

Izlaz

```
s1 = hello, s2 = hello
same heap data: false
```

Premeštanje: s2 preuzima vlasništvo



Slika 6: Move: s1 više nije validan, s2 preuzima vlasništvo nad podacima

Premeštanje: brojevi se kopiraju, String se premešta

primeri/ownership/src/bin/strings_move.rs

```rust
fn main() {
    // Integers implement the Copy trait, so the value is copied
    let x = 5;
    let y = x;
    println!("{}", x);
    println!("{}", y);

    // Strings do not implement the Copy trait, so the value is moved
```

```
    let s1 = String::from("hello");

    let s2 = s1;
    // let s2 = s1.clone();
    // println!("{s1}, world!");
    println!("{s2}, world!");
}
```

Izlaz

```
5
5
hello, world!
```

Premeštanje: korišćenje posle premeštanja (greška)

primeri/ownership/src/bin/nocompile_move.rs

```
fn main() {
    let s1 = String::from("hello");

    let s2 = s1;

    println!("{s1}, world!"); // Error: `s1` was moved to `s2`
    println!("{s2}");
}
```

Izlaz

```
error[E0382]: borrow of moved value: `s1`
 --> ownership/src/bin/nocompile_move.rs:6:16
  |
2 |     let s1 = String::from("hello");
  |         -- move occurs because `s1` has type `String`, which does not
 ↪  implement the `Copy` trait
3 |
4 |     let s2 = s1;
  |              -- value moved here
5 |
6 |     println!("{s1}, world!"); // Error: `s1` was moved to `s2`
  |               ^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
4 |     let s2 = s1.clone();
  |                ++++++++

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ownership` (bin "nocompile_move") due to 1
 ↪  previous error
```

Vlasništvo: Copy tipovi

- Tip koji implementira osobinu `Copy`: pri dodeli se vrednost kopira bit po bit, a original ostaje važeći.
- Tip sa `Copy` ne sme da implementira `Drop`.
- Odlučuje osobina `Copy`, a ne to da li je vrednost na steku ili na heapu.
- Primeri:

  - Prosti tipovi (celi i realni brojevi, `bool`, `char`) i `()`.
  - Torke i nizovi čiji su svi elementi `Copy`.
  - Deljene reference `&T`.

Vlasništvo: tipovi koji se premeštaju

- Tip bez `Copy`: pri dodeli se vlasništvo prenosi (premešta), a original više nije važeći.
- Kopija se pravi eksplicitno metodom `clone()` (osobina `Clone`).
- Primeri:
  - Tipovi sa heap alokacijom: `String`, `Vec<T>`, `Box<T>`.
  - Pametni pokazivači i ćelije: `Rc<T>`, `Arc<T>`, `Cell<T>`, `RefCell<T>`.
  - Promenljive reference `&mut T`.
  - Sopstveni tipovi (strukture, enumeracije), osim ako implementiraju `Copy`.
  - Torke i nizovi koji sadrže bar jedan tip bez `Copy`.

Torke: Copy i premeštanje (greška)

primeri/ownership/src/bin/nocompile_tuple.rs

```
fn take_tuple_ints(t: (i32, i32)) {
    println!("Tuple contains: {}, {}", t.0, t.1);
}

fn take_tuple_int_string(t: (i32, String)) {
    println!("Tuple contains: {}, {}", t.0, t.1);
}

fn main() {
    let tup = (42, 99);
    take_tuple_ints(tup); // `tup` is copied, not moved, since both `i32`
 ↪  values implement `Copy`.
    println!("{}", tup.1); // This works because `tup` wasn't moved.

    let tup = (42, String::from("hello")); // In Rust, tuples will only
     ↪  implement Copy if all of their elements implement Copy. Since
     ↪  String does not implement Copy, the tuple (String, i32) as a whole
     ↪  does not implement Copy
    take_tuple_int_string(tup); // Ownership of the entire tuple (and thus
 ↪  the String inside it) is moved to the function.
    println!("{}", tup.1); // This doesnt work because `tup` was moved.
}
```

Izlaz

```
error[E0382]: borrow of moved value: `tup`
  --> ownership/src/bin/nocompile_tuple.rs:16:20
   |
14 |     let tup = (42, String::from("hello")); // In Rust, tuples will
 ↪  only implement Copy if all of their elements implement Copy. Sinc...
   |         --- move occurs because `tup` has type `(i32, String)`, which
 ↪  does not implement the `Copy` trait
15 |     take_tuple_int_string(tup); // Ownership of the entire tuple (and
 ↪  thus the String inside it) is moved to the function.
   |                           --- value moved here
16 |     println!("{}", tup.1); // This doesnt work because `tup` was
 ↪  moved.
   |                     ^^^^^ value borrowed here after move
   |
```

```
note: consider changing this parameter type in function
 ↪  `take_tuple_int_string` to borrow instead if owning the value isn't
 ↪  necessary
  --> ownership/src/bin/nocompile_tuple.rs:5:29
   |
 5 | fn take_tuple_int_string(t: (i32, String)) {
   |    ---------------------    ^^^^^^^^^^^^^ this parameter takes
 ↪  ownership of the value
   |    |
   |    in this function
help: consider cloning the value if the performance cost is acceptable
   |
15 |     take_tuple_int_string(tup.clone()); // Ownership of the entire
 ↪  tuple (and thus the String inside it) is moved to the function.
   |                              ++++++++

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ownership` (bin "nocompile_tuple") due to 1
 ↪  previous error
```

Vlasništvo: premeštanje u funkciju

primeri/ownership/src/bin/move_into_function.rs

```rust
fn main() {
    let s = String::from("hello"); // s comes into scope

    takes_ownership(s); // s's value moves into the function...
                        // ... and so is no longer valid here

    let x = 5; // x comes into scope

    makes_copy(x); // x would move into the function,
                   // but i32 is Copy, so it's okay to still
                   // use x afterward
} // Here, x goes out of scope, then s. But because s's value was moved,
 ↪  nothing
  // special happens.

fn takes_ownership(some_string: String) {
    // some_string comes into scope
    println!("{some_string}");
} // Here, some_string goes out of scope and `drop` is called. The backing
  // memory is freed.

fn makes_copy(some_integer: i32) {
    // some_integer comes into scope
    println!("{some_integer}");
} // Here, some_integer goes out of scope. Nothing special happens.
```

Izlaz

```
hello
5
```

Vlasništvo: povratne vrednosti

primeri/ownership/src/bin/move_return_values.rs

```rust
fn main() {
    let s1 = gives_ownership(); // gives_ownership moves its return
                                // value into s1

    let s2 = String::from("hello"); // s2 comes into scope

    let s3 = takes_and_gives_back(s2); // s2 is moved into
                                       // takes_and_gives_back, which also
                                       // moves its return value into s3
} // Here, s3 goes out of scope and is dropped. s2 was moved, so nothing
  // happens. s1 goes out of scope and is dropped.

fn gives_ownership() -> String {
    // gives_ownership will move its
    // return value into the function
    // that calls it

    let some_string = String::from("yours"); // some_string comes into
    ↪  scope

    some_string // some_string is returned and
                // moves out to the calling
                // function
}

// This function takes a String and returns one
fn takes_and_gives_back(a_string: String) -> String {
    // a_string comes into
    // scope

    a_string // a_string is returned and moves out to the calling function
}
```

Izlaz

Vlasništvo: vraćanje vlasništva torkom

primeri/ownership/src/bin/move_tuple_return.rs

```rust
fn main() {
    let s1 = String::from("hello");

    let (s2, len) = calculate_length(s1);

    println!("The length of '{s2}' is {len}.");
}

fn calculate_length(s: String) -> (String, usize) {
    let length = s.len(); // len() returns the length of a String

    (s, length)
}
```

Izlaz

```
The length of 'hello' is 5.
```

Reference

Reference i pozajmljivanje

- Problem prenosa vlasništva.
  - Lečiti vraćanjem vrednosti iz funkcija.
  - Sprečiti korišćenjem referenci.

Reference

- Nalik pokazivačima u standardnim programskim jezicima.
- Vlasnik je neko drugi, s tim da i ostali (oni kojima je pozajmljeno) mogu pristupiti vrednosti.
- Garantovano validna vrednost.
- Dozvoljeno:
  - Ili 1 mutabilna referenca,
  - Ili 1 do N nemutabilnih referenci.
- Referenca važi do svoje poslednje upotrebe, a ne do kraja bloka (*non-lexical lifetimes*).

Pozajmljivanje: reference na vrednosti

primeri/ownership/src/bin/borrow.rs

```rust
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("The length of '{s1}' is {len}.");

    let i1 = 5;
    let i2 = &i1;
    let i3 = *i2; // Value is copied
    println!("The value of i1 is {i1}.");
    println!("The value of i2 is {i2}.");
    println!("The value of i3 is {i3}.");
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

Izlaz

```
The length of 'hello' is 5.
The value of i1 is 5.
The value of i2 is 5.
The value of i3 is 5.
```

Reference: nepromenljiva i promenljiva referenca

primeri/ownership/src/bin/reference_mutability.rs

```rust
fn main() {
    let mut s = String::from("hello");

    // Immutable reference: the value can be read, but not modified
    read(&s);
```

```
    // Mutable reference: the value can be modified
    change(&mut s);
    println!("{s}");
}

fn read(some_string: &String) {
    println!("{some_string}");
    // some_string.push_str(", world"); // Error: cannot borrow as
     ↪  mutable, it is behind a `&` reference
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

Izlaz

```
hello
hello, world
```

Reference: dve promenljive reference (greška)

primeri/ownership/src/bin/nocompile_mut_twice.rs

```
fn main() {
    let mut s = String::from("hello");

    let r1 = &mut s;
    let r2 = &mut s; // Error: second mutable borrow while `r1` is alive

    println!("{r1}, {r2}");
}
```

Izlaz

```
error[E0499]: cannot borrow `s` as mutable more than once at a time
 --> ownership/src/bin/nocompile_mut_twice.rs:5:14
  |
4 |     let r1 = &mut s;
  |              ------ first mutable borrow occurs here
5 |     let r2 = &mut s; // Error: second mutable borrow while `r1` is
 ↪  alive
  |              ^^^^^^ second mutable borrow occurs here
6 |
7 |     println!("{r1}, {r2}");
  |                -- first borrow later used here

For more information about this error, try `rustc --explain E0499`.
error: could not compile `ownership` (bin "nocompile_mut_twice") due to 1
 ↪  previous error
```

Reference: više nepromenljivih i jedna promenljiva

primeri/ownership/src/bin/references_shared_then_mut.rs

```
fn main() {
    let mut s = String::from("hello");
```

```
    let r1 = &s;
    let r2 = &s;
    println!("{r1} and {r2}");

    // This ability, to have multiple unmutable references and mutable
     ↪  references in the same scope, without overlapping use is called
     ↪  Non-lexical lifetimes.
    let r3 = &mut s;
    println!("{r3}");
}
```

Izlaz

```
hello and hello
hello
```

Reference: čitanje i izmena istovremeno (greška)

primeri/ownership/src/bin/nocompile_shared_and_mut.rs

```
fn main() {
    let mut s = String::from("hello");

    let r1 = &s;
    let r2 = &mut s; // Error: mutable borrow while `r1` is still in use

    println!("{r1}, {r2}");
}
```

Izlaz

```
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as
 ↪  immutable
 --> ownership/src/bin/nocompile_shared_and_mut.rs:5:14
  |
4 |     let r1 = &s;
  |              -- immutable borrow occurs here
5 |     let r2 = &mut s; // Error: mutable borrow while `r1` is still in
 ↪  use
  |              ^^^^^^ mutable borrow occurs here
6 |
7 |     println!("{r1}, {r2}");
  |                -- immutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `ownership` (bin "nocompile_shared_and_mut") due
 ↪  to 1 previous error
```

Reference: mut na imenu i &mut u tipu

	`&T`	`&mut T`
`let r`	čita vrednost	menja vrednost
`let mut r`	čita vrednost, može da pokazuje na drugu vrednost	menja vrednost, može da pokazuje na drugu vrednost

- `mut` na imenu: referenca može da pokazuje na drugu vrednost.

- `&mut` u tipu: kroz referencu se vrednost može menjati.

Reference: &mut od promenljive bez mut (greška)

primeri/ownership/src/bin/nocompile_mut_borrow_immutable.rs

```rust
fn main() {
    let x = 5;

    let r = &mut x; // Error: `x` is not declared as mutable
    *r += 1;

    println!("{x}");
}
```

Izlaz

```
error[E0596]: cannot borrow `x` as mutable, as it is not declared as
 ↪  mutable
 --> ownership/src/bin/nocompile_mut_borrow_immutable.rs:4:13
  |
4 |     let r = &mut x; // Error: `x` is not declared as mutable
  |             ^^^^^^ cannot borrow as mutable
  |
help: consider changing this to be mutable
  |
2 |     let mut x = 5;
  |         +++

For more information about this error, try `rustc --explain E0596`.
error: could not compile `ownership` (bin
 ↪  "nocompile_mut_borrow_immutable") due to 1 previous error
```

Reference: mut parametar i &mut parametar

primeri/ownership/src/bin/mut_parameter.rs

```rust
// `mut s` makes only the binding inside the function mutable;
// the caller's variable is moved in and does not need `mut`
fn add_suffix(mut s: String) -> String {
    s.push_str("!");
    s
}

// `&mut String` changes the caller's value in place
fn add_suffix_in_place(s: &mut String) {
    s.push_str("!");
}

fn main() {
    let a = String::from("hello");
    let b = add_suffix(a);
    println!("{b}");

    let mut c = String::from("hello");
    add_suffix_in_place(&mut c);
    println!("{c}");
}
```

Izlaz

```
hello!
hello!
```

Reference: promenljivost vezivanja i reference

primeri/ownership/src/bin/references_bindings.rs

```rust
fn main() {
    // 1. Immutable binding to an immutable reference
    let x = 10; // `x` is an immutable binding to the value 10
    let y = &x; // `y` is an immutable reference to `x`
    println!("y (immutable binding to immutable reference): {}", y);

    // Attempting to reassign `y` would cause a compile-time error:
    // y = &20; // Error: cannot assign to `y` because it is an immutable
     ↪  binding

    // Attempting to modify the value of `x` through `y` would also cause
     ↪  an error:
    // *y = 20; // Error: cannot assign to `*y` which is behind an `&`
     ↪  reference

    // 2. Mutable binding to an immutable reference
    let mut y_ref = &x; // `y_ref` is a mutable binding to an immutable
     ↪  reference of `x`
    println!(
        "y_ref (initial mutable binding to immutable reference): {}",
        y_ref
    );

    // Since `y_ref` is a mutable binding, we can reassign it to point to
     ↪  a different reference:
    y_ref = &20; // Allowed: `y_ref` can be reassigned to a different
 ↪  reference
    println!("y_ref (reassigned to new immutable reference): {}", y_ref);

    // However, we still cannot modify the value of `x` through `y_ref`:
    // *y_ref = 20; // Error: cannot assign to `*y_ref` which is behind an
     ↪  immutable reference

    // 3. Immutable binding to a mutable reference
    let mut z = 30; // `z` is a mutable binding to the value 30
    let z_ref = &mut z; // `z_ref` is an immutable binding to a mutable
     ↪  reference of `z`
    *z_ref += 10; // Allowed: modifies `z` through `z_ref`
    println!(
        "z (after modification through immutable binding to mutable
         ↪  reference): {}",
        z
    );

    // Attempting to reassign `z_ref` to another mutable reference would
     ↪  cause an error:
    // z_ref = &mut z; // Error: cannot assign to immutable variable
     ↪  `z_ref`

    // 4. Mutable binding to a mutable reference
```

```
    let mut a = 40; // `a` is a mutable binding to the value 40
    let mut a_ref = &mut a; // `a_ref` is a mutable binding to a mutable
     ↪  reference of `a`
    *a_ref += 10; // Allowed: modifies `a` through `a_ref`
    println!(
        "a (after modification through mutable binding to mutable
         ↪  reference): {}",
        a
    );

    // Since `a_ref` is a mutable binding, we can reassign it to point to
     ↪  another mutable reference:
    a_ref = &mut a; // Allowed: `a_ref` can be reassigned to a new mutable
 ↪  reference
    *a_ref += 10; // Allowed: modifies `a` again through `a_ref`
    println!(
        "a (after reassignment and modification through mutable binding to
         ↪  mutable reference): {}",
        a
    );
}
```

Izlaz

```
y (immutable binding to immutable reference): 10
y_ref (initial mutable binding to immutable reference): 10
y_ref (reassigned to new immutable reference): 20
z (after modification through immutable binding to mutable reference): 40
a (after modification through mutable binding to mutable reference): 50
a (after reassignment and modification through mutable binding to mutable
 ↪  reference): 60
```

Reference: preusmeravanje na drugi element

primeri/ownership/src/bin/references_reassign.rs

```
fn main() {
    let mut v: Vec<i32> = vec![1, 2, 3];

    let mut num: &mut i32 = &mut v[2];
    // let num: &mut i32 = &mut v[2];

    num = &mut v[1];

    *num += 1;

    num = &mut v[2];

    println!("Third element is {}", *num);

    println!("Vector is now {:?}", v);
}
```

Izlaz

```
Third element is 3
Vector is now [1, 3, 3]
```

Reference: ponovno pozajmljivanje

primeri/ownership/src/bin/references_reborrow.rs

```rust
fn main() {
    let mut v: Vec<i32> = vec![1, 2, 3];

    // We borrow the reference to the third element of the vector.
    let num: &mut i32 = &mut v[2];
    // We dereference the reference to the third element of the vector,
     ↪  and we create immutable reference to it.
    let num2: &i32 = &*num;

    // *num dereferences the &mut i32 to i32.
    // &*num then takes an immutable reference to that i32.
    // This is called "reborrowing": you create a shared (&i32) borrow
     ↪  from a mutable (&mut i32) borrow.
    // After this point, the compiler treats num as frozen for mutation
     ↪  while num2 is in use, but you can still read through num.

    // This in turn means that the num reference to the third element of
     ↪  the vector is immutable.
    // *num += 1;
    println!("{} {}", *num, *num2);
}
```

Izlaz

```
3 3
```

Reference: &mut se premešta pri dodeli (greška)

primeri/ownership/src/bin/nocompile_mut_reference_move.rs

```rust
fn main() {
    let mut s = String::from("hello");

    let r = &mut s;
    let r2 = r; // `&mut String` is not `Copy`: `r` is moved to `r2`

    r2.push_str(", world");
    r.push_str("!"); // Error: borrow of moved value: `r`

    println!("{s}");
}
```

Izlaz

```
error[E0382]: borrow of moved value: `r`
 --> ownership/src/bin/nocompile_mut_reference_move.rs:8:5
  |
4 |     let r = &mut s;
  |         - move occurs because `r` has type `&mut String`, which does
 ↪  not implement the `Copy` trait
...
7 |     r2.push_str(", world");
  |        ------------------- `r` moved due to this method call
8 |     r.push_str("!"); // Error: borrow of moved value: `r`
  |     ^ value borrowed here after move
  |
```

```
note: `std::string::String::push_str` takes ownership of the receiver
 ↪  `self`, which moves `r`
 --> /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/alloc/src/
 ↪  string.rs:1104:25

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ownership` (bin "nocompile_mut_reference_move")
 ↪  due to 1 previous error
```

Reference: &mut prosleđena funkciji dvaput

- `&mut` nije `Copy`, pa bi se `r` pri prosleđivanju premestio.
- Kad je očekivani tip poznat kao `&mut T`, kompajler umesto premeštanja pravi privremenu pozajmicu (*reborrow*): `r` je posle poziva ponovo upotrebljiv.
  - Bezbedno je, jer je `r` zamrznut dok pozajmica traje, pa je i dalje aktivna samo jedna `&mut`.
- Isto važi za polje strukture sa deklarisanim tipom, `let` sa eksplicitnim tipom i poziv metode.
- Ne važi kad tip nije poznat: generički parametar `T`, `let` bez tipa, `vec![r]`. Tamo se `r` premešta.

primeri/ownership/src/bin/mut_reference_pass_twice.rs

```rust
fn add_exclamation(s: &mut String) {
    s.push('!');
}

fn main() {
    let mut s = String::from("hello");

    let r = &mut s;

    // `r` is not moved: the parameter type is `&mut String`,
    // so each call reborrows it
    add_exclamation(r);
    add_exclamation(r);

    println!("{r}");
}
```

Izlaz

```
hello!!
```

Reference: operator * i poziv metode

primeri/ownership/src/bin/references_deref.rs

```rust
fn main() {
    let mut n = 10;
    let r = &mut n;

    // Assignment and arithmetic need an explicit dereference
    *r += 1;
    println!("{}", *r + 1);
    // r += 1; // Error: cannot apply `+=` to `&mut {integer}`

    let mut s = String::from("hello");
```

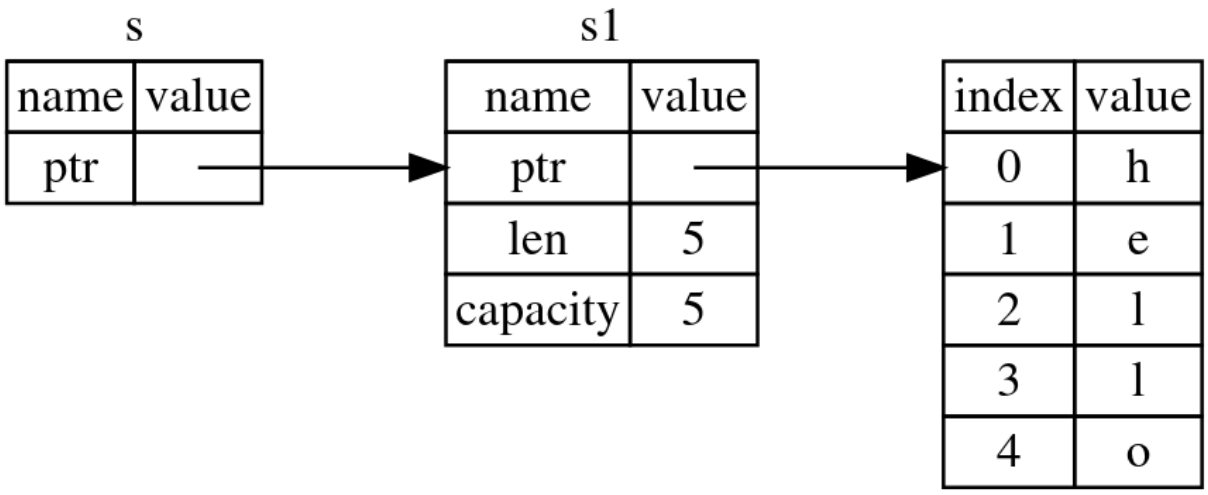
```
    let rs = &mut s;

    // A method call dereferences automatically
    rs.push_str("!");
    (*rs).push_str("?"); // the same, written explicitly
    println!("{rs}");
}
```

Izlaz

```
12
hello!?
```

Reference: s pokazuje na String s1



Slika 7: Referenca s na String s1, koji pokazuje na podatke na heapu

Pointer Safety Principle

Data should never be aliased and mutated at the same time.

Viseći pokazivači

- Pokazivač koji pokazuje na memoriju koja je možda data nekome drugom.
- Viseće reference nije moguće napraviti.
- Primer:

primeri/ownership/src/bin/dangling.rs

```
fn main() {
    // let reference_to_nothing = dangle();
    let reference_to_something = no_dangle();
}

// fn dangle() -> &str {
//     let s = String::from("hello");

//     &s
// }

fn no_dangle() -> String {
```

```
    let s = String::from("hello");

    s
}
```

Izlaz

```
```

Slice tip

- Omogućava referisanje na deo kolekcije (Vector/String/...)
  - Umesto na celu kolekciju.
  - Za String je uobičajen nepromenljiv slice `&str`.
  - Promenljiv slice `&mut str` postoji, ali ne može da menja dužinu (npr. `make_ascii_uppercase`).
  - Granice slice-a su pozicije u bajtovima (UTF-8), a ne u karakterima: isecanje mora da počne i završi na granici karaktera, inače program izaziva paniku.

Slice: ručno traženje prve reči

primeri/ownership/src/bin/manual_str.rs

```
fn first_word(s: &String) -> usize {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        println!("{}, {}", i, item); // Unicode is backwards-compatible
        ↪ with ASCII :)
        if item == b' ' {
            return i;
        }
    }

    s.len()
}

fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s); // word will get the value 5

    s.clear(); // this empties the String, making it equal to ""

    // word still has the value 5 here, but there's no more string that
    // we could meaningfully use the value 5 with. word is now totally
    ↪ invalid
}
```

Izlaz

```
0, 104
1, 101
2, 108
3, 108
4, 111
5, 32
```

Slice: prva reč kao slice i izmena Stringa (greška)

primeri/ownership/src/bin/nocompile_slice_then_clear.rs

```rust
fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}

fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s); // `word` is a slice of `s`: it borrows `s`

    // first_word("hello"); // Would not compile: expects `&String`

    s.clear(); // Error: `s` is still borrowed by `word`

    println!("{word}");
}
```

Izlaz

```
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as
 ↪  immutable
  --> ownership/src/bin/nocompile_slice_then_clear.rs:20:5
   |
16 |     let word = first_word(&s); // `word` is a slice of `s`: it
 ↪  borrows `s`
   |                           -- immutable borrow occurs here
...
20 |     s.clear(); // Error: `s` is still borrowed by `word`
   |     ^^^^^^^^^ mutable borrow occurs here
21 |
22 |     println!("{word}");
   |                ---- immutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `ownership` (bin "nocompile_slice_then_clear")
 ↪  due to 1 previous error
```

Slice: parametar &str umesto &String

primeri/ownership/src/bin/str_slice.rs

```rust
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
```

```
    }

    &s[..]
}

fn main() {
    let my_string = String::from("hello world");

    // `first_word` works on slices of `String`s, whether partial or whole
    let word = first_word(&my_string[0..6]);
    let word = first_word(&my_string[..]);
    // `first_word` also works on references to `String`s, which are
     ↪  equivalent
    // to whole slices of `String`s
    let word = first_word(&my_string);
    let word = first_word("zdravo svete");

    let my_string_literal = "hello world";

    // `first_word` works on slices of string literals, whether partial or
     ↪  whole
    let word = first_word(&my_string_literal[0..6]);
    let word = first_word(&my_string_literal[..]);

    // Because string literals *are* string slices already,
    // this works too, without the slice syntax!
    let word = first_word(my_string_literal);
}
```

Izlaz

```
```

Slice: isecanje stringa sa ne-ASCII slovima (panika)

primeri/ownership/src/bin/str_slice_utf8.rs

```
fn main() {
    let word = String::from("šuma");

    println!("{}", &word[0..2]);
    println!("{}", &word[0..1]); // Panic: byte index 1 is not a char
     ↪  boundary
}
```

Izlaz

```
š

thread 'main' (7767) panicked at ownership/src/bin/str_slice_utf8.rs:5:25:
end byte index 1 is not a char boundary; it is inside 'š' (bytes 0..2 of
 ↪  string)
note: run with `RUST_BACKTRACE=1` environment variable to display a
 ↪  backtrace
```

Slice: slice niza

primeri/ownership/src/bin/slices.rs

```
fn main() {
    let a = [1, 2, 3, 4, 5];

    let slice = &a[1..3];

    assert_eq!(slice, &[2, 3]);

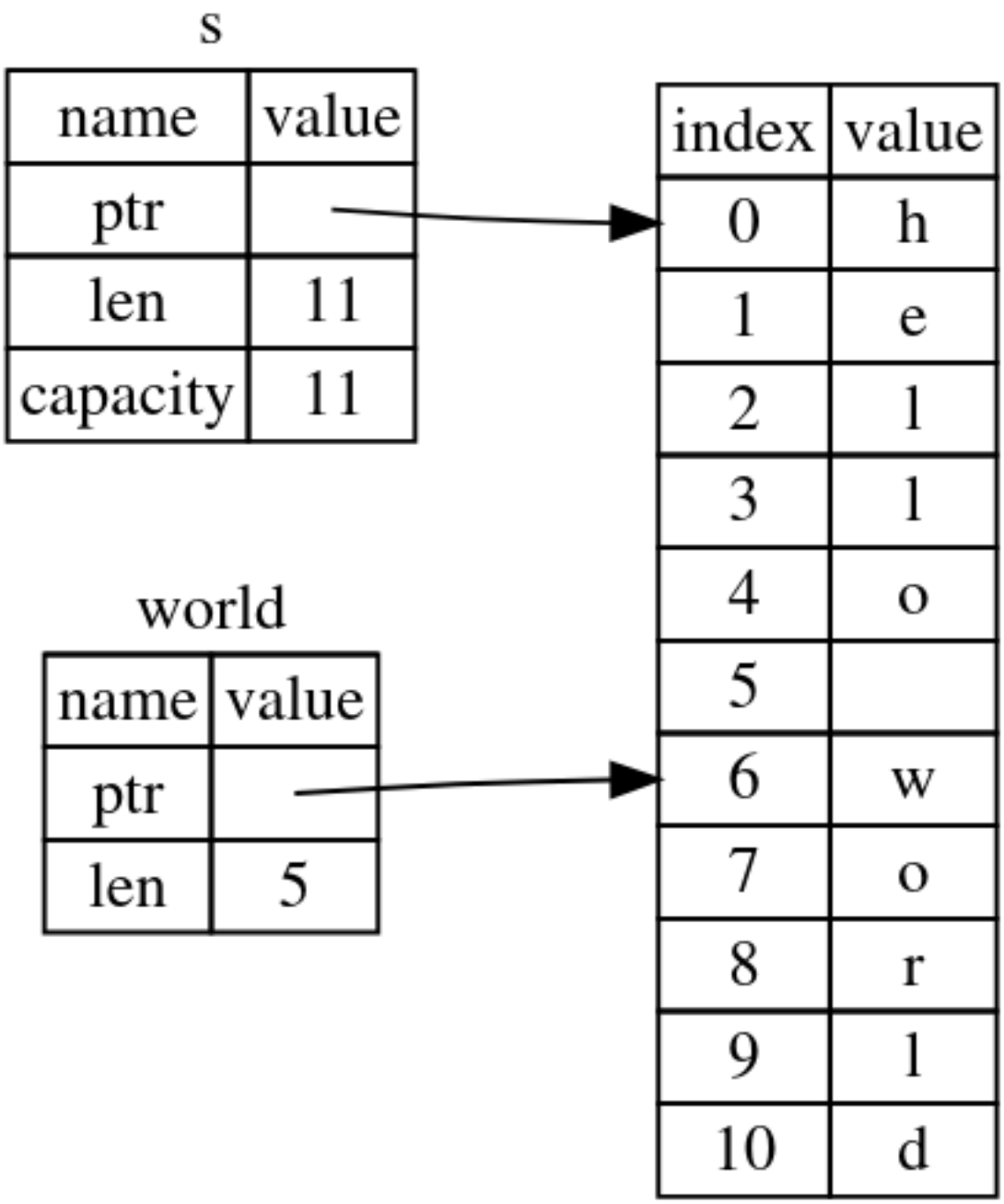
    analyze_slice(slice);
}

fn analyze_slice(slice: &[i32]) {
    println!("First element: {}", slice[0]);
}
```

Izlaz

```
First element: 2
```

Slice: world pokazuje na deo stringa s



Slika 8: Slice: world pokazuje na deo stringa s

Iteracija: for petlja nad vektorom

primeri/ownership/src/bin/iterations.rs

```rust
fn main() {
    let words = vec![String::from("one"), String::from("two")];

    // Iterating over the vector immutably
    for word in &words {
        println!("{}", word);
```

```
    }

    let mut words = vec![String::from("one"), String::from("two")];

    // Iterating over the vector mutably
    for word in &mut words {
        word.push('!');
    }
    println!("{:?}", words);

    let words = vec![String::from("one"), String::from("two")];

    // Consuming the vector: each `String` is moved out into `word`
    for word in words {
        println!("{}", word);
    }
    // `words` can no longer be used: its elements were moved out
    // println!("{:?}", words); // Error: borrow of moved value: `words`
}
```

Izlaz

```
one
two
["one!", "two!"]
one
two
```

Iteracija: torke u vektoru

primeri/ownership/src/bin/iterations_tuples.rs

```
fn main() {
    // A vector of tuples, where each tuple contains a String and an i32
    let mut people = vec![
        (String::from("Alice"), 30),
        (String::from("Bob"), 25),
        (String::from("Charlie"), 35),
    ];

    // Example 1: Consuming the collection (moving ownership)
    consume_tuples(people.clone());

    // Example 2: Borrowing the tuples immutably (read-only)
    borrow_immutable_tuples(&people);

    // Example 3: Borrowing the tuples mutably (modifying in place)
    borrow_mutable_tuples(&mut people);

    // Verify the changes after mutable borrow
    println!("Modified people vector:");
    for (name, age) in &people {
        println!("Name: {}, Age: {}", name, age);
    }
}

// Consuming the collection by taking ownership of each tuple
fn consume_tuples(people: Vec<(String, i32)>) {
```

```
    println!("\nConsuming the collection:");
    for (name, age) in people {
        println!("Name: {}, Age: {}", name, age); // Ownership of name and
         ↪  age is moved here
    }
    // The vector `people` is now consumed and cannot be used further
}

// Borrowing the collection immutably (read-only)
fn borrow_immutable_tuples(people: &Vec<(String, i32)>) {
    println!("\nBorrowing immutably:");
    for (name, age) in people.iter() {
        println!("Name: {}, Age: {}", name, age); // Borrowed, so we can
         ↪  still use the original vector
    }
    // The original `people` vector can still be used after this function
     ↪  because we only borrowed it
}

// Borrowing the collection mutably to modify the values
fn borrow_mutable_tuples(people: &mut Vec<(String, i32)>) {
    println!("\nBorrowing mutably (modifying in place):");
    for (name, age) in people.iter_mut() {
        *age += 1; // Increment age by 1 for each person
        println!("Modified Name: {}, New Age: {}", name, age);
    }
    // The original `people` vector is modified because we borrowed it
     ↪  mutably
}
```

Izlaz

```
Consuming the collection:
Name: Alice, Age: 30
Name: Bob, Age: 25
Name: Charlie, Age: 35

Borrowing immutably:
Name: Alice, Age: 30
Name: Bob, Age: 25
Name: Charlie, Age: 35

Borrowing mutably (modifying in place):
Modified Name: Alice, New Age: 31
Modified Name: Bob, New Age: 26
Modified Name: Charlie, New Age: 36
Modified people vector:
Name: Alice, Age: 31
Name: Bob, Age: 26
Name: Charlie, Age: 36
```

Iteracija: metode iter, iter_mut i into_iter

primeri/ownership/src/bin/vector.rs

```
fn main() {
    let mut vec = vec![10, 20, 30];
```

```
    // Immutable borrow: iter() does not move the vector, so it can be
     ↪  used afterwards.
    let borrowed_sum: i32 = vec.iter().sum();
    println!("Sum (borrowed): {}, vector: {:?}", borrowed_sum, vec);

    // Mutable borrow: iter_mut() modifies the elements in place.
    // (`for item in &mut vec` is shorthand for `for item in
     ↪  vec.iter_mut()`.)
    for item in vec.iter_mut() {
        *item += 10;
    }
    println!("Updated vector: {:?}", vec);

    // Consuming the vector: into_iter() takes ownership, `vec` cannot be
     ↪  used after this.
    let sum: i32 = vec.into_iter().sum();
    println!("Sum of elements (consumed): {}", sum);
    // println!("{:?}", vec); // Error: borrow of moved value `vec`
}
```

Izlaz

```
Sum (borrowed): 60, vector: [10, 20, 30]
Updated vector: [20, 30, 40]
Sum of elements (consumed): 90
```

Vektor: premeštanje elementa iz vektora (greška)

- Vektor je vlasnik elemenata: element tipa `String` se ne može premestiti iz njega po indeksu.
- Umesto toga: pozajmi (`&v[0]`), kopiraj (`v[0].clone()`) ili izvadi (`v.remove(0)`).

primeri/ownership/src/bin/nocompile_move_from_vec.rs

```
fn main() {
    let v = vec![String::from("hello"), String::from("world")];

    let first = v[0]; // Error: cannot move out of index of `Vec<String>`

    println!("{first}");
}
```

Izlaz

```
error[E0507]: cannot move out of index of `Vec<String>`
 --> ownership/src/bin/nocompile_move_from_vec.rs:4:17
  |
4 |     let first = v[0]; // Error: cannot move out of index of
 ↪  `Vec<String>`
  |                 ^^^^ move occurs because value has type `String`,
 ↪  which does not implement the `Copy` trait
  |
help: consider borrowing here
  |
4 |     let first = &v[0]; // Error: cannot move out of index of
 ↪  `Vec<String>`
  |                 +
help: consider cloning the value if the performance cost is acceptable
  |
```

```
4 |     let first = v[0].clone(); // Error: cannot move out of index of
 ↪  `Vec<String>`
  |                     ++++++++

For more information about this error, try `rustc --explain E0507`.
error: could not compile `ownership` (bin "nocompile_move_from_vec") due
 ↪  to 1 previous error
```

HashMap: pozajmljivanje i vlasništvo pri iteraciji

primeri/ownership/src/bin/hashmap.rs

```rust
use std::collections::HashMap;

fn main() {
    let mut map = HashMap::new();

    map.insert(String::from("Alice"), 50); // String is moved.
    map.insert(String::from("Bob"), 30);

    // Borrowing key-value pairs
    for (key, value) in &map {
        println!("{}: {}", key, value); // Borrowing the map immutably.
    }

    // Removing an entry
    let bob_score = map.remove("Bob"); // Entry is consumed (moved out).
    println!("Removed Bob's score: {:?}", bob_score);

    // Not consuming the map
    let keys_iter = map.keys();
    let keys: Vec<&String> = keys_iter.collect();
    println!("Keys: {:?}", keys);

    // Consuming the map
    let keys_iter = map.into_keys();
    let keys: Vec<String> = keys_iter.collect();
    println!("Keys: {:?}", keys);
}
```

Izlaz

```
Bob: 30
Alice: 50
Removed Bob's score: Some(30)
Keys: ["Alice"]
Keys: ["Alice"]
```

Zadaci

Zadatak 1

Napisati program u Rust programskom jeziku koji unetu rečenicu deli u vektor reči.

Zadatak 2

Napisati program u Rust programskom jeziku koji obrće elemente u okviru niza na licu mesta (in-place).

Zadatak 3

Napisati program u Rust programskom jeziku koji sortira vektor označenih brojeva koristeći bubble sort algoritam.

Zadatak 4

Napisati program u Rust programskom jeziku koji sortira vektor označenih brojeva koristeći quick sort algoritam.

Zadatak 5

Napisati program u Rust programskom jeziku koji prima unos korisnika u obliku rečenice, a zatim:

- Broji broj reči u rečenici
- Pronalazi najdužu i najkraću reč u rečenici
- Proverava da li je rečenica palindrom ili ne

Izvori

Izvori

- Rust Community. “The Rust Programming Language - the Rust Programming Language.” Rust-Lang.org, 2018, doc.rust-lang.org/book/.
- Rust Team. “Rust Programming Language.” Rust-Lang.org, 2018, www.rust-lang.org/.
- Rust Community. “Tour of Rust - Let’s Go on an Adventure!” Tourofrust.com, tourofrust.com/.
- Crichton, Will. “Experiment Introduction - the Rust Programming Language.” Brown.edu, rust-book.cs.brown.edu/.